217. Contains Duplicate Easy

Acceptance: 64.0% | LeetCode

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: nums = [1,2,3,1]

Output: true

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: nums = [1,2,3,4]

Output: false

Explanation:

All elements are distinct.

Example 3:

Input: nums = [1,1,1,3,3,4,3,2,4,2]

Output: true

Constraints:

- `1 <= nums.length <= 105`
- `-109 <= nums[i] <= 109`

Claude Opus Solution

217. Contains Duplicate — Full Breakdown

1. 💡 Intuition

Key Pattern: Membership Tracking with a Hash Set

The core question is: "*Have I seen this number before?*"

Every time we visit a number, we need to answer that question in **O(1)**. A **hash set** is the perfect data structure for this — it stores unique elements and supports O(1) average-case lookup and insertion.

> This is a classic **"track what you've seen"** pattern. Any time a problem asks about duplicates, repeated elements, or previously-visited values → think **hash set** first.

The brute-force alternative (check every pair) is $O(n^2)$ — too slow for $n=10^5$. Sorting brings it to O(n log n). But a hash set gets us to **O(n) time with a single pass**.

2. 🗺️ Approach (Pseudocode)

```
function containsDuplicate(nums):
    seen = empty set

    for each number in nums:
        if number is already in seen:
            return True          ← duplicate found immediately
        add number to seen       ← mark as visited

    return False                 ← no duplicates found after full scan
```

Why this works: A set only stores *unique* values. If we try to "re-add" something we've already seen, we detect it *before* adding and immediately return `True`. We never need to scan backwards or compare pairs.

3. 🐍 Python Solution

```python
from typing import List

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:

        # ──────────────────────────────────────────────────────
        # WHY a set?
        #   - Hash sets give O(1) average for both 'in' checks and .add()
        #   - We only care about EXISTENCE of a number, not its count
```

```
        #   - So a set (not a dict) is the minimal sufficient structure
        # ──────────────────────────────────────────────────────────
        seen = set()

        for number in nums:

            # STEP 1: Before processing this number, check if we've
            # seen it before. If yes → duplicate confirmed, exit early.
            # Early exit is crucial: on [1,1,9,9,...] we stop at index 1,
            # not after scanning all n elements.
            if number in seen:
                return True

            # STEP 2: First time seeing this number — record it so
            # future iterations can detect if it appears again.
            seen.add(number)

        # STEP 3: We exhausted the entire array without finding any
        # number that appeared twice → all elements are distinct.
        return False
```

---

4. 🔬 Walkthrough on Example 1

Input: `nums = [1, 2, 3, 1]`

Step	`number`	`seen` before check	`number in seen` ?	Action
1	`1`	`{}`	❌ No	add `1` → `{1}`
2	`2`	`{1}`	❌ No	add `2` → `{1,2}`
3	`3`	`{1,2}`	❌ No	add `3` → `{1,2,3}`
4	`1`	`{1,2,3}`	✅ **Yes!**	**return `True`**

We return at step 4 without processing the rest of the array. ✅

---

5. 📊 Complexity Analysis

✅ Optimal Solution (Hash Set)

	Complexity	Reasoning
Time	**O(n)**	Single pass through the array. Each `in` check and `.add()` is O(1) average for a hash set.
Space	**O(n)**	In the worst case (no duplicates), we store all `n` elements in the set.

---

6. 🔄 Alternative Approaches

Approach 1: Brute Force — O(n²) time, O(1) space

```
# Compare every pair (i, j) where j > i
# Correct but TLEs on large inputs (n = 10⁵ → 10¹⁰ operations)
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] == nums[j]:
            return True
return False
```

✅ No extra space | ❌ Way too slow

Approach 2: Sort then Scan — O(n log n) time, O(1) space

```
# After sorting, any duplicate MUST be adjacent
# So a single linear scan of neighbors suffices
nums.sort()
for i in range(1, len(nums)):
    if nums[i] == nums[i - 1]:
        return True
return False
```

✅ Better than brute force | ❌ Mutates the input array, slower than hash set

---

Approach 3: One-liner Set Comparison — O(n) time, O(n) space

```
# If the set (unique elements) is smaller than the array → duplicates exist
return len(set(nums)) < len(nums)
```

✅ Concise | ❌ No early exit — always processes all `n` elements even if duplicate is at index 0

🏆 Summary

Approach	Time	Space	Early Exit?	Mutates Input?
Brute Force	O(n²)	O(1)	✅	❌
Sort + Scan	O(n log n)	O(1)*	✅	✅
Hash Set ⭐	**O(n)**	**O(n)**	✅	❌
Set One-liner	O(n)	O(n)	❌	❌

The hash set approach is the clear winner — optimal time complexity, doesn't mutate input, and exits as soon as the first duplicate is found.

Codex Solution

1. Intuition

We need to know whether **any number appears more than once**.

A very common pattern for this type of problem is:

> Use a **hash set** to remember what we have already seen.

As we scan through `nums`:

- If the current number is already in the set, then we found a duplicate.
- Otherwise, add it to the set and continue.

This works because set lookup is average `O(1)` time.

---

2. Approach

Pseudocode

```
create an empty set called seen

for each number in nums:
    if number is already in seen:
        return true
    add number to seen

return false
```

---

3. Python Solution

```
from typing import List

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        # This set will store numbers we have already visited.
        seen = set()

        # Iterate through every number in the input array.
        for num in nums:
            # If num is already in the set, we have seen it before.
            # Therefore, a duplicate exists.
            if num in seen:
```

```
                return True

            # Otherwise, record this number as seen.
            seen.add(num)

        # If we finish the loop without finding a duplicate,
        # then all elements are distinct.
        return False
```

---

4. Complexity Analysis

Let `n` be the length of `nums`.

Time Complexity: `O(n)`

We loop through the array once.

For each number:

- Checking if it exists in a set takes average `O(1)`
- Adding it to a set also takes average `O(1)`

So the total time complexity is:

```
O(n)
```

Space Complexity: `O(n)`

In the worst case, all elements are distinct, so we store all `n` numbers in the set.

Therefore, the space complexity is:

```
O(n)
```

---

5. Alternative Approaches

Approach 1: Compare Length With Set Length

Python makes this problem very concise:

```python
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(nums) != len(set(nums))
```

If there are duplicates, converting `nums` to a set removes them, so the lengths will be different.

Complexity:

- Time: `O(n)`
- Space: `O(n)`

Approach 2: Sorting

We can sort the array first. After sorting, duplicates will be next to each other.

```python
from typing import List

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        nums.sort()

        for i in range(1, len(nums)):
            if nums[i] == nums[i - 1]:
                return True

        return False
```

Complexity:

- Time: `O(n log n)` because of sorting
- Space: `O(1)` extra space if sorting in-place, ignoring sorting internals

This approach modifies the original array.

Approach 3: Brute Force

Compare every pair of elements.

```python
from typing import List

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] == nums[j]:
                    return True

        return False
```

Complexity:

- Time: `O(n^2)`
- Space: `O(1)`

This is simple but too slow for `nums.length <= 10^5`.

Key Pattern

This problem is a classic example of:

> **Use a hash set to detect duplicates in one pass.**

Whenever you need to check whether something has appeared before, a set or hash table is often the best tool.